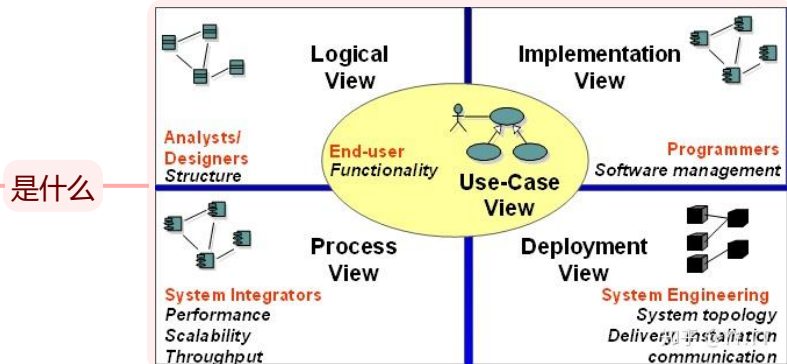


软件方法

2. 架构图

实例
详情
5视图



是什么

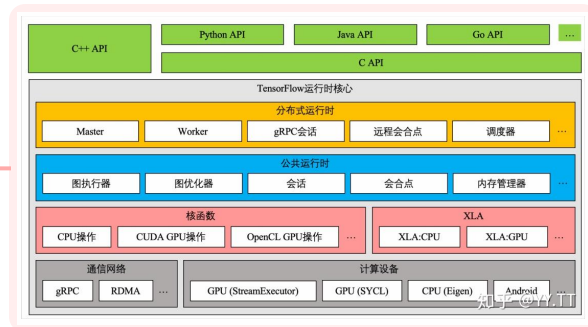
4+1视图

说明(Lidup)

逻辑视图 (Logical view)

对系统职责的的逐级划分

例



实现视图 (Implemention view)

主要包括两部分信息

对逻辑架构元素，描述其代码位置，可以是代码仓位置，或代码目录，或是开源软件的版本信息等

系统的构建，即如何将代码编译成二进制交付件（比如.so/.bin）。这个构建信息需要包括构建依赖、构建工具链、构建环境信息

部署视图 (Deployment view)

也称为物理视图

物理环境可能是服务器、PC机、移动终端等物理设备

软件环境可以是虚拟机、容器、进程或线程

处理流程视图 (Process view)

中文称为过程视图、运行视图或处理视图

逻辑视图、开发视图和部署视图，描述的都是系统的静态信息，而运行视图就是用来描述系统中的动态信息的。运行视图最常见的设计工具就是UML的序列图。

场景视图(Use-Case view)

DDD

是什么

Domain-Driven Design, 即领域驱动设计, 是在一定领域内, 发现问题, 抽象共性不变的流程, 进而提供问题解决方案的过程。

方法

通过一个统一语言 (ubiquitous language) 领域建模、领域划分等一系列手段来降低复杂度, 并基于面向对象分析 (OOP) 技术进行了分层规划, 对软件开发全生命周期使用语言进行统一, 并强调业务与技术相结合的一种过程。

为什么要建模

建模通过可视化的模型, 让懂业务的人和懂技术的人可以互相交流沟通。同时, 我们做了一个模型之后可以很方便的复用传播, 在其它项目中进行改造, 最后还可以对做出的决策进行文档化。

什么是“领域”模型

领域的逻辑就是显性的专业知识。

比如现在需要开发一款法务人员使用的软件系统, 那么就会存在这样的问题: 法务人员不懂软件开发, 开发人员不懂法律条文信息, 而在开发过程中二者又需要进行不断的沟通和确认, 法律领域的模型就是连接二者的一个桥梁, 领域模型主要倾向于将领域的知识给描述出来, 管理合规性。

实现DDD的落地



沟通

3F

事实信息 (FACT)
感受和判断 (FEELING)
意图 (FOCUS)

通过逻辑视图、开发视图加部署视图, 我们已经可以知道系统中每一个逻辑架构元素、每一份代码, 最终会运行在什么位置上。反向也可以通过运行环境上, 找到所有其上运行的逻辑架构元素和代码。